

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer and Communication Engineering (CCE)

HOTEL RESERVATION SYSTEM

A Relational Database Design & SQL Implementation Project

Submitted for: Database Management Systems (DBMS) Course Project

Group Members

Name	ID / Role	Contribution
Abdur Rahman	E241018	Database design, SQL implementation, documentation
Ramiatul Amin Chowdhury	E241030	ER diagram, normalization, testing
Minhajun Noor Miraj	E241027	Sample data, reports, presentation

Course Code : CCE-3506

Course Title : Database Management Systems Sessional

Supervised by: Md. Hasan Shorif

Institution: International Islamic University Chittagong (IIUC)

Table of Contents

Table of Contents.....	2
1. Introduction	4
2. Problem Statement.....	4
3. Objectives	4
4. Scope	4
5. Features.....	5
6. Assumptions	5
7. ER Diagram	5
Relationship summary	6
8. Relational Schema	7
Hotels.....	7
RoomTypes.....	7
Rooms.....	7
Guests	7
Staff	8
Reservations.....	8
Payments.....	8
Services.....	8
ReservationServices (junction table)	8
Reviews	9
ReservationAudit (supports the DELETE trigger).....	9
9. Normalization	9
1NF — First Normal Form	9
2NF — Second Normal Form.....	9
3NF — Third Normal Form	9
10. SQL Implementation	10
10.1 Database & table creation	10
10.2 Foreign keys and referential actions	10
10.3 Sample data.....	10
10.4 Update and delete statements.....	10
11. Sample Data.....	11
12. SQL Queries	11
12.1 Simple queries	11
12.2 Aggregate functions.....	11
12.3 Joins.....	11
12.4 Subqueries	11
12.5 Views.....	12
12.6 Functions	12
12.7 Stored procedures	12
12.8 Triggers.....	12

13. Reports.....	12
14. Output Screenshots	12
15. Challenges	13
16. Conclusion.....	13
17. References	13

1. Introduction

The hospitality industry depends on efficient reservation management to provide quality service and maximize room occupancy. Traditional paper-based reservation systems often lead to booking errors, duplicated reservations, difficulty in tracking customer information, and inefficient room management.

The **Hotel Reservation System** is a relational database project developed using SQL to automate the hotel booking process. The system stores and manages information about hotels, rooms, customers, reservations, payments, staff, and customer reviews. It allows hotel staff to efficiently handle bookings, monitor room availability, manage payments, and generate useful reports for decision-making.

This project applies Database Management System concepts including database design, normalization, relational schema, SQL implementation, constraints, joins, views, functions, stored procedures, and reports

2. Problem Statement

Many small and medium-sized hotels still rely on manual methods to manage reservations and customer information. These methods can result in:

- Double booking of rooms
- Loss of customer information
- Incorrect payment records
- Difficulty in checking room availability
- Time-consuming report generation
- Poor reservation management
- Data inconsistency and redundancy

A computerized Hotel Reservation System solves these problems by maintaining accurate, secure, and well-organized records using a relational database.

3. Objectives

The system is built to:

- Store and manage information for multiple hotel properties, their rooms and room types.
- Record guest details and their complete reservation history.
- Track reservations from booking through check-in, check-out or cancellation.
- Record payments (supporting part-payments) against each reservation.
- Offer optional paid services (breakfast, laundry, spa, etc.) attached to a reservation.
- Collect guest reviews per hotel.
- Produce management reports: revenue, occupancy, top guests, best-selling services, staff cost.
- Demonstrate, with working SQL, every concept required by the course: constraints, joins, aggregates, subqueries, views, functions, procedures and triggers.

4. Scope

In scope: room and hotel inventory management, guest records, the reservation lifecycle, payments, optional services, and guest reviews, together with reporting views for hotel management.

Out of scope: online payment gateway integration, a front-end / web interface, multi-currency support, and channel-manager integration with third-party booking sites (e.g. Booking.com). These are natural extensions but are not required to demonstrate the DBMS concepts targeted by this project.

5. Features

The proposed database system provides the following features:

- Store hotel branch information
- Store different room categories
- Maintain room details
- Customer registration
- Room booking
- Reservation status management
- Payment management
- Staff information management
- Customer review management
- Search available rooms
- Generate monthly reservation reports
- Generate payment summaries
- Track customer booking history
- Calculate hotel revenue
- Prevent duplicate room bookings

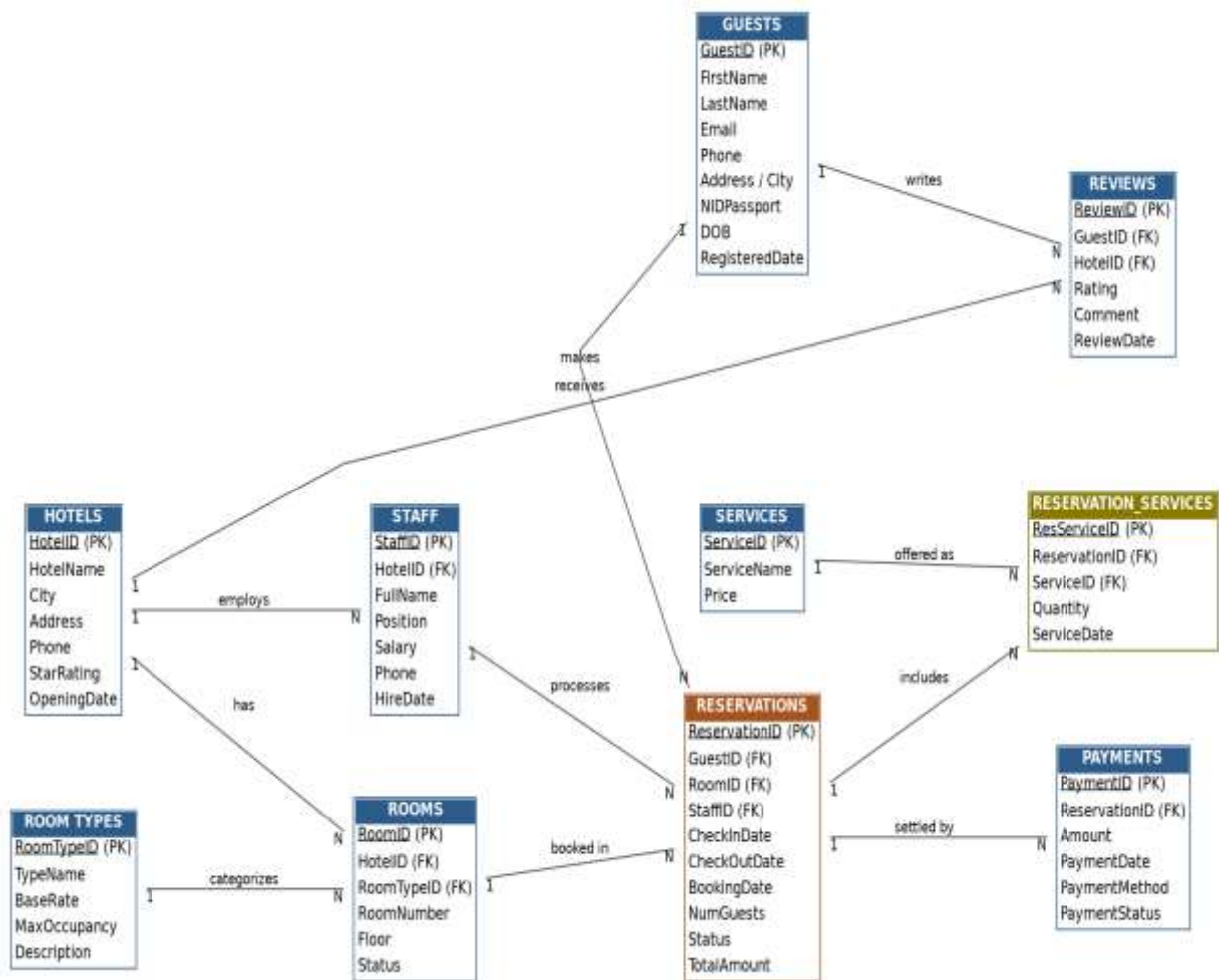
6. Assumptions

The following assumptions are made during system design:

- One customer can make multiple reservations.
- One reservation belongs to only one customer.
- One room belongs to only one hotel.
- One room has one room type.
- One reservation is made for one room.
- One reservation has one payment.
- One customer can submit multiple reviews.
- One staff member belongs to one hotel.
- Every reservation has valid check-in and check-out dates.
- A room cannot be booked by two customers for the same reservation.

7. ER Diagram

The diagram below shows all ten entities, their primary keys (PK), foreign keys (FK) and the cardinality of every relationship. It was produced with Graphviz from the schema definition; the same diagram can equally be redrawn in Draw.io, Lucidchart or MySQL Workbench.



Relationship summary

Relationship	Cardinality	Meaning
Hotels — Rooms	1 : N	One hotel has many rooms
RoomTypes — Rooms	1 : N	One room type applies to many rooms
Hotels — Staff	1 : N	One hotel employs many staff
Hotels — Reviews	1 : N	One hotel receives many reviews
Guests — Reservations	1 : N	One guest makes many reservations
Guests — Reviews	1 : N	One guest writes many reviews
Rooms — Reservations	1 : N	One room is booked in many reservations (over time)
Staff — Reservations	1 : N	One staff member processes many reservations
Reservations — Payments	1 : N	One reservation can be settled by several payments
Reservations — Services	M : N	Resolved via the ReservationServices junction table

8. Relational Schema

The ER model above converts directly into the following 11 relational tables (10 core entities + 1 audit table used by a trigger). PK = Primary Key, FK = Foreign Key.

Hotels

Column	Type	Constraint
HotelID	INT	PK, AUTO_INCREMENT
HotelName	VARCHAR(100)	NOT NULL
City	VARCHAR(50)	NOT NULL
Address	VARCHAR(150)	NOT NULL
Phone	VARCHAR(20)	NOT NULL, UNIQUE
StarRating	TINYINT	DEFAULT 3, CHECK 1–5
OpeningDate	DATE	NOT NULL

RoomTypes

Column	Type	Constraint
RoomTypeID	INT	PK, AUTO_INCREMENT
TypeName	VARCHAR(50)	NOT NULL, UNIQUE
BaseRate	DECIMAL(10,2)	NOT NULL, CHECK > 0
MaxOccupancy	TINYINT	DEFAULT 2, CHECK > 0
Description	VARCHAR(255)	

Rooms

Column	Type	Constraint
RoomID	INT	PK, AUTO_INCREMENT
HotelID	INT	FK -> Hotels
RoomTypeID	INT	FK -> RoomTypes
RoomNumber	VARCHAR(10)	NOT NULL
Floor	TINYINT	DEFAULT 1
Status	ENUM	Available/Occupied/Maintenance, DEFAULT Available

Guests

Column	Type	Constraint
GuestID	INT	PK, AUTO_INCREMENT
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	NOT NULL, UNIQUE
Phone	VARCHAR(20)	NOT NULL
Address / City	VARCHAR(150)/(50)	
NIDPassport	VARCHAR(30)	UNIQUE
DOB	DATE	

RegisteredDate	DATE	DEFAULT CURRENT_DATE
----------------	------	----------------------

Staff

Column	Type	Constraint
StaffID	INT	PK, AUTO_INCREMENT
HotelID	INT	FK -> Hotels
FullName	VARCHAR(100)	NOT NULL
Position	VARCHAR(50)	NOT NULL
Salary	DECIMAL(10,2)	CHECK >= 0
Phone	VARCHAR(20)	
HireDate	DATE	NOT NULL

Reservations

Column	Type	Constraint
ReservationID	INT	PK, AUTO_INCREMENT
GuestID	INT	FK -> Guests
RoomID	INT	FK -> Rooms
StaffID	INT	FK -> Staff (nullable)
CheckInDate / CheckOutDate	DATE	NOT NULL, CHECK CheckOut > CheckIn
BookingDate	DATE	DEFAULT CURRENT_DATE
NumGuests	TINYINT	DEFAULT 1, CHECK > 0
Status	ENUM	Confirmed/CheckedIn/CheckedOut/Cancelled
TotalAmount	DECIMAL(10,2)	Auto-filled by trigger

Payments

Column	Type	Constraint
PaymentID	INT	PK, AUTO_INCREMENT
ReservationID	INT	FK -> Reservations
Amount	DECIMAL(10,2)	NOT NULL, CHECK > 0
PaymentDate	DATE	DEFAULT CURRENT_DATE
PaymentMethod	ENUM	Cash/Card/MobileBanking/BankTransfer
PaymentStatus	ENUM	Pending/Completed/Refunded

Services

Column	Type	Constraint
ServiceID	INT	PK, AUTO_INCREMENT
ServiceName	VARCHAR(50)	NOT NULL, UNIQUE
Price	DECIMAL(8,2)	NOT NULL, CHECK >= 0

ReservationServices (junction table)

Column	Type	Constraint
--------	------	------------

ResServiceID	INT	PK, AUTO_INCREMENT
ReservationID	INT	FK -> Reservations
ServiceID	INT	FK -> Services
Quantity	INT	DEFAULT 1, CHECK > 0
ServiceDate	DATE	DEFAULT CURRENT_DATE

Reviews

Column	Type	Constraint
ReviewID	INT	PK, AUTO_INCREMENT
GuestID	INT	FK -> Guests
HotelID	INT	FK -> Hotels
Rating	TINYINT	CHECK 1–5
Comment	VARCHAR(255)	
ReviewDate	DATE	DEFAULT CURRENT_DATE

ReservationAudit (supports the DELETE trigger)

Column	Type	Constraint
AuditID	INT	PK, AUTO_INCREMENT
ReservationID	INT	NOT NULL
GuestID	INT	NOT NULL
ActionType	VARCHAR(20)	NOT NULL
ActionDate	DATETIME	DEFAULT CURRENT_TIMESTAMP

9. Normalization

Every table in the schema is normalized to Third Normal Form (3NF):

1NF — First Normal Form

A table is in 1NF when every column holds a single, atomic value and there are no repeating groups. All 11 tables satisfy this: for example, a guest's phone number is one value in one column, not a comma-separated list, and the many-to-many relationship between reservations and services is not squeezed into a repeating column — it is broken out into its own ReservationServices table.

2NF — Second Normal Form

A table is in 2NF when it is in 1NF and every non-key column depends on the whole primary key, not just part of it. This only matters for tables with a composite key. Every table here uses a single-column surrogate key (an auto-increment ID), so there is no composite key for a column to partially depend on — 2NF is satisfied automatically. The one true composite relationship, Reservations-to-Services, is resolved with its own surrogate key (ResServiceID) in the ReservationServices junction table, so even that relationship is fully 2NF-compliant.

3NF — Third Normal Form

A table is in 3NF when it is in 2NF and no non-key column depends on another non-key column (no transitive dependency). This is enforced throughout the design: a room stores only a RoomTypeID, not the room type's name or nightly rate — those live only in RoomTypes and are looked up through the foreign key. A reservation stores GuestID and RoomID, not the guest's phone number or the room's hotel name. The one deliberate exception is Reservations.TotalAmount, which is a derived value (nights x rate) stored for reporting speed; this is a conscious, documented denormalization, kept consistent automatically by the trg_before_reservation_insert trigger rather than left for the application layer to maintain, so it does not create a real data-integrity risk.

10. SQL Implementation

The complete, runnable script is attached as `hotel_reservation_system.sql`. It is organized in the same order as this report: database creation, table creation (DDL), sample data (DML), update/delete statements, then all query categories, views, functions, procedures and triggers required by the assignment. Below is a short walk-through of the key techniques used; every numbered requirement in the brief is implemented at least once (usually several times) in the attached file.

10.1 Database & table creation

```
CREATE DATABASE HotelReservationSystem;
USE HotelReservationSystem;

CREATE TABLE Hotels (
    HotelID      INT AUTO_INCREMENT PRIMARY KEY,
    HotelName    VARCHAR(100) NOT NULL,
    City         VARCHAR(50)  NOT NULL,
    Phone        VARCHAR(20)  NOT NULL UNIQUE,
    StarRating   TINYINT NOT NULL DEFAULT 3 CHECK (StarRating BETWEEN 1 AND 5),
    OpeningDate  DATE NOT NULL
);
```

Every table in the script uses this same pattern: an `AUTO_INCREMENT` primary key, `NOT NULL` on required columns, `UNIQUE` where a real-world value must not repeat (phone numbers, emails, service names), `CHECK` constraints for valid ranges, and `DEFAULT` values so day-to-day inserts do not have to repeat obvious values.

10.2 Foreign keys and referential actions

```
CREATE TABLE Rooms (
    RoomID      INT AUTO_INCREMENT PRIMARY KEY,
    HotelID     INT NOT NULL,
    RoomTypeID  INT NOT NULL,
    RoomNumber  VARCHAR(10) NOT NULL,
    Status       ENUM('Available', 'Occupied', 'Maintenance') DEFAULT 'Available',
    CONSTRAINT fk_rooms_hotel FOREIGN KEY (HotelID) REFERENCES Hotels(HotelID) ON DELETE CASCADE,
    CONSTRAINT fk_rooms_roomtype FOREIGN KEY (RoomTypeID) REFERENCES RoomTypes(RoomTypeID),
    CONSTRAINT uq_room_per_hotel UNIQUE (HotelID, RoomNumber)
);
```

`ON DELETE CASCADE` is used where a child row genuinely has no meaning without its parent (a room without a hotel, a reservation without a guest). `ON DELETE SET NULL` is used for `Reservations.StaffID`, since a reservation should not disappear just because the staff member who processed it later leaves.

10.3 Sample data

The database is populated with 179 realistic rows across 11 tables: 5 hotels, 5 room types, 30 rooms, 20 guests, 15 staff, 30 reservations, 25 payments, 8 services, 26 service add-ons and 15 reviews — comfortably above the assignment's minimums of 6 tables, 15+ rows per main table and 80+ rows overall.

10.4 Update and delete statements

6 `UPDATE` statements and 4 `DELETE` statements are included (minimums: 5 and 3). Examples:

```
UPDATE Staff SET Salary = Salary * 1.05 WHERE Position = 'Receptionist';

DELETE FROM Reservations WHERE Status = 'Cancelled';
```

11. Sample Data

A representative excerpt of the seed data is shown below; the full INSERT statements for all 11 tables are in the attached SQL file.

HotelID	HotelName	City	StarRating
1	Sea Pearl Beach Resort	Cox's Bazar	5
2	Radisson Blu Chattogram	Chattogram	5
3	Pan Pacific Sonargaon	Dhaka	5
4	Grand Sultan Tea Resort	Sylhet	4
5	Best Western Sylhet Heights	Sylhet	3

GuestID	FirstName	LastName	City	RegisteredDate
1	Abdur	Rahman	Chattogram	2023-01-10
2	Minhajun	Miraj	Chattogram	2023-01-12
3	Ramiatul	Chowdhury	Chattogram	2023-01-15
10	John	Smith	London	2023-03-11

12. SQL Queries

All query categories required by Part F are implemented in the attached script. One representative example from each category is shown here for readability; the full set (5+ simple, 5+ aggregate, 8+ joins including INNER/LEFT/RIGHT/multi-table, 3+ subqueries, 2+ views, 2+ functions, 2+ procedures, 2 triggers) is in hotel_reservation_system.sql.

12.1 Simple queries

```
SELECT r.RoomID, r.RoomNumber, h.HotelName, rt.TypeName, rt.BaseRate
FROM Rooms r
JOIN Hotels h ON r.HotelID = h.HotelID
JOIN RoomTypes rt ON r.RoomTypeID = rt.RoomTypeID
WHERE r.Status = 'Available'
ORDER BY rt.BaseRate ASC;
```

12.2 Aggregate functions

```
SELECT h.HotelName, ROUND(AVG(rv.Rating),2) AS AvgRating, COUNT(rv.ReviewID) AS NumReviews
FROM Hotels h
JOIN Reviews rv ON h.HotelID = rv.HotelID
GROUP BY h.HotelID, h.HotelName
ORDER BY AvgRating DESC;
```

12.3 Joins

```
-- RIGHT JOIN: every service, with reservations that used it (0 if unused)
SELECT sv.ServiceName, rs.ReservationID, rs.Quantity
FROM ReservationServices rs
RIGHT JOIN Services sv ON rs.ServiceID = sv.ServiceID
ORDER BY sv.ServiceName;
```

12.4 Subqueries

```
-- Rooms that have never been booked
SELECT RoomID, RoomNumber
FROM Rooms
WHERE RoomID NOT IN (SELECT DISTINCT RoomID FROM Reservations);
```

12.5 Views

```
CREATE OR REPLACE VIEW vw_HotelRevenueSummary AS
SELECT h.HotelID, h.HotelName,
       COUNT(DISTINCT res.ReservationID) AS TotalReservations,
       COALESCE(SUM(p.Amount), 0)        AS TotalRevenue
FROM Hotels h
LEFT JOIN Rooms r ON h.HotelID = r.HotelID
LEFT JOIN Reservations res ON r.RoomID = res.RoomID
LEFT JOIN Payments p ON res.ReservationID = p.ReservationID AND p.PaymentStatus =
'Completed'
GROUP BY h.HotelID, h.HotelName;
```

12.6 Functions

```
CREATE FUNCTION fn_CalculateStayCost(p_RoomID INT, p_CheckIn DATE, p_CheckOut DATE)
RETURNS DECIMAL(10,2) DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_Rate DECIMAL(10,2);
    SELECT rt.BaseRate INTO v_Rate FROM Rooms r
        JOIN RoomTypes rt ON r.RoomTypeID = rt.RoomTypeID WHERE r.RoomID = p_RoomID;
    RETURN v_Rate * DATEDIFF(p_CheckOut, p_CheckIn);
END;
```

12.7 Stored procedures

```
CALL sp_MakeReservation(1, 2, 1, '2026-09-01', '2026-09-04', 2);
CALL sp_CancelReservation(28);
```

12.8 Triggers

```
CREATE TRIGGER trg_before_reservation_insert
BEFORE INSERT ON Reservations
FOR EACH ROW
BEGIN
    SET NEW.TotalAmount = fn_CalculateStayCost(NEW.RoomID, NEW.CheckInDate,
NEW.CheckOutDate);
END;
```

13. Reports

11 management reports are implemented (minimum required: 10):

- R1. Total revenue and reservation count per hotel
- R2. Top 5 highest-spending guests
- R3. Most popular room type by number of bookings
- R4. Currently available rooms, hotel-wise
- R5. Monthly revenue report
- R6. Staff salary report, hotel-wise
- R7. Reservation status summary (confirmed / cancelled / checked-out / etc.)
- R8. Hotel-wise average guest review rating
- R9. Best-selling extra service by quantity and revenue
- R10. Repeat guests (more than one reservation)
- R11. Room occupancy rate per hotel

14. Output Screenshots

HotelName	City	StarRating
Sea Pearl Beach Resort	Cox's Bazar	5
Radisson Blu Chattogram	Chattogram	5
Pan Pacific Sonargaon	Dhaka	5

HotelName	FullName	Position
Best Western Sylhet Heights		
Grand Sultan Tea Resort		
Pan Pacific Sonargaon		
Radisson Blu Chattogram		
Sea Pearl Beach Resort		

15. Challenges

- Deciding how to price a reservation without duplicating data: solved by keeping the rate only on RoomTypes and deriving TotalAmount automatically with a trigger + function, instead of storing (and risking desynchronized) prices on every room or reservation.
- Handling the many-to-many relationship between reservations and optional services: solved with a dedicated ReservationServices junction table rather than a repeating column.
- Keeping room availability accurate as reservations are cancelled or checked out: solved with an AFTER UPDATE trigger that automatically frees the room the moment a reservation's status changes.
- Avoiding accidental data loss on delete: solved with an AFTER DELETE trigger that logs every deleted reservation to a ReservationAudit table before it is gone for good.

16. Conclusion

The **Hotel Reservation System** successfully demonstrates the use of Database Management System concepts in a real-world scenario. The database is well-structured, normalized, and efficient for managing hotels, rooms, customers, reservations, and payments. This project improved our practical knowledge of SQL, database design, and data management, and it can serve as a strong foundation for building a complete hotel management application in the future.

17. References

- MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- Elmasri, R. & Navathe, S. B., Fundamentals of Database Systems, 7th Edition, Pearson.
- Silberschatz, A., Korth, H. F. & Sudarshan, S., Database System Concepts, 7th Edition, McGraw-Hill.
- Graphviz — Graph Visualization Software, <https://graphviz.org/>